

ASSIGNMENT NO:-06

Name:-Rajdeep Das

Roll no:- CSB25203

Subject:- Advanced Programming

```
import java.util.*;
import java.util.stream.Collectors;

class Student {

    private int id;
    private String name;
    private List<String> courses;
    private Map<String, Integer> scores;

    public Student(int id, String name,
                  List<String> courses,
                  Map<String, Integer> scores) {
        this.id = id;
        this.name = name;
        this.courses = new ArrayList<>(courses);
        this.scores = new HashMap<>(scores);
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public List<String> getCourses() { return courses; }
```

```
publicMap<String, Integer> getScores() { return scores; }
```

```
publicdouble getAverageScore() {  
    return scores.values()  
        .stream()  
        .mapToInt(Integer::intValue)  
        .average()  
        .orElse(0.0);  
}  
}
```

```
publicclass StudentPerformanceAnalyzer {
```

```
//——Core methods——

---


```

```
publicstatic List<Student> sortByAverageDescending(List<Student> students) {  
    return students.stream()  
        .sorted(Comparator.comparingDouble(Student::getAverageScore).reversed())  
        .collect(Collectors.toList());  
}
```

```
publicstatic List<Student> getTopNStudents(List<Student> students, int n) {  
    return sortByAverageDescending(students)  
        .stream()  
        .limit(n)  
        .collect(Collectors.toList());  
}
```

```
publicstatic Map<String, Double> getAverageScorePerCourse(List<Student> students) {  
    Set<String> allCourses = getAllUniqueCourses(students);  
    Map<String, Double> averageMap = new HashMap<>();
```

```

for(Stringcourse : allCourses) {
    doubleavg = students.stream()
        .mapToInt(s -> s.getScores().getOrDefault(course, 0))
        .average()
        .orElse(0.0);
    averageMap.put(course, avg);
}
returnaverageMap;
}

```

```

publicstaticSet<String> getAllUniqueCourses(List<Student> students) {
    returnstudents.stream()
        .flatMap(s -> s.getCourses().stream())
        .collect(Collectors.toCollection(HashSet::new));
}

```

```

publicstaticList<Student> getPassedStudents(List<Student> students) {
    returnstudents.stream()
        .filter(s-> s.getScores().values().stream().allMatch(score -> score >= 40))
        .collect(Collectors.toList());
}

```

//——Random data generation———

```

privatestaticfinal String[] NAMES = {
    "Nitish","Subro", "Aman", "Rajdeep",
    "Rishav","Ankit", "Atul", "Ayush"
};

```

```

privatestaticfinal String[] COURSE_POOL = {
    "Math","Physics", "Chemistry", "Biology",

```

```

        "English", "History", "Computer Science", "Economics"
    };

    /**
     *Generates a student with a given name and random courses/scores.
     */
    public static Student generateStudentWithName(int id, String name, Random rng) {
        List<String> coursePool = new ArrayList<>(Arrays.asList(COURSE_POOL));
        Collections.shuffle(coursePool, rng);
        int numCourses = 3 + rng.nextInt(3); // 3, 4, or 5
        List<String> courses = new ArrayList<>(coursePool.subList(0, numCourses));

        Map<String, Integer> scores = new HashMap<>();
        for(String course : courses) {
            scores.put(course, 20 + rng.nextInt(81)); // marks: 20-100
        }

        return new Student(id, name, courses, scores);
    }

    /**
     *Generates n students using UNIQUE names (no repetition within the list).
     *Shuffles the name pool so order is random each run.
     */
    public static List<Student> generateRandomStudents(int n, Random rng) {
        List<Student> students = new ArrayList<>();
        List<String> shuffled = new ArrayList<>(Arrays.asList(NAMES));
        Collections.shuffle(shuffled, rng);
        for(int i = 0; i < n; i++) {
            String name = shuffled.get(i % shuffled.size());
            students.add(generateStudentWithName(i + 1, name, rng));
        }
    }

```

```

    }

    return students;
}

//——Empirical complexity analysis over 1000 random cases ——

public static void analyzeCourseAverageComplexity() {

    System.out.println("\n===== EMPIRICAL COMPLEXITY ANALYSIS: Course Averages (1000
randomcases) =====");

    Random rng = new Random(42);
    int[] sizes = {10, 50, 100, 500, 1000};
    int trials = 1000;

    System.out.printf("%-12s %-20s %-20s\n", "Students(n)", "Avg Time (ns)", "Scaled (ns/n)");
    System.out.println("-".repeat(54));

    for(int n : sizes) {
        long totalTime = 0;
        for(int t = 0; t < trials; t++) {
            List<Student> students = generateRandomStudents(n, rng);
            long start = System.nanoTime();
            getAverageScorePerCourse(students);
            totalTime += System.nanoTime() - start;
        }
        long avgNs = totalTime / trials;
        System.out.printf("%-12d %-20d %-20.2f\n", n, avgNs, (double) avgNs / n);
    }

    System.out.println("\nTheoretical: O(n·m + k·n) where k=unique courses, m=courses per
student");

    System.out.println("Worst Case : O(n2·m)");
}

```

```

        System.out.println("Space    :O(k)");
    }

    public static void analyzeSortingComplexity() {
        System.out.println("\n===== EMPIRICAL COMPLEXITY ANALYSIS: Sorting Top-N Students
(1000 random cases) =====");

        Random rng = new Random(99);
        int[] sizes = {10, 50, 100, 500, 1000};
        int trials = 1000;

        System.out.printf("%-12s %-20s %-20s%n", "Students(n)", "Avg Time (ns)", "Scaled
(ns/n·logn)");
        System.out.println("-".repeat(56));

        for(int n : sizes) {
            long totalTime = 0;
            for(int t = 0; t < trials; t++) {
                List<Student> students = generateRandomStudents(n, rng);
                long start = System.nanoTime();
                getTopNStudents(students, 2);
                totalTime += System.nanoTime() - start;
            }
            long avgNs = totalTime / trials;
            double nlogn = n * (Math.log(n) / Math.log(2));
            System.out.printf("%-12d %-20d %-20.2f%n", n, avgNs, avgNs / nlogn);
        }

        System.out.println("\nTheoretical: O(n·m + n·log n) where m=courses per student");
        System.out.println("If m const : O(n·log n)");
        System.out.println("Space    : O(n)");
    }

```

```
//-----Main-----
```

```
public static void main(String[] args) {
```

```
    Random rng = new Random();
```

```
    //Pick 6 unique names from the pool (shuffle and take first 6)
```

```
    List<String> nameList = new ArrayList<>(Arrays.asList(NAMES));
```

```
    Collections.shuffle(nameList, rng);
```

```
    List<Student> students = new ArrayList<>();
```

```
    for(int i=0; i < 6; i++) {
```

```
        students.add(generateStudentWithName(i + 1, nameList.get(i), rng));
```

```
    }
```

```
    System.out.println("==== 6 RANDOMLY GENERATED STUDENTS =====");
```

```
    for(Student s : students) {
```

```
        System.out.println("\nID : " + s.getId());
```

```
        System.out.println("Name : " + s.getName());
```

```
        System.out.println("Scores: " + s.getScores());
```

```
    }
```

```
    System.out.println("\n==== Students Sorted by Average (Descending) =====");
```

```
    sortByAverageDescending(students)
```

```
        .forEach(s -> System.out.printf("%-12s Avg = %.2f%n",
```

```
            s.getName(), s.getAverageScore()));
```

```
    System.out.println("\n==== Top 2 Students =====");
```

```
    getTopNStudents(students, 2)
```

```
        .forEach(s -> System.out.printf("%-12s Avg = %.2f%n",
```

```
            s.getName(), s.getAverageScore()));
```

```

System.out.println("\n===== Average Score Per Course =====");
getAverageScorePerCourse(students)
    .forEach((course, avg) ->
        System.out.printf("%-20s : %.2f%n", course, avg));

System.out.println("\n===== All Unique Courses =====");
System.out.println(getAllUniqueCourses(students));

System.out.println("\n===== Passed Students (all scores >= 40) =====");
List<Student> passed = getPassedStudents(students);
if(passed.isEmpty()) {
    System.out.println("None");
} else {
    passed.forEach(s -> System.out.println(s.getName()));
}

analyzeCourseAverageComplexity();
analyzeSortingComplexity();
}
}

```